LibTorch : Day 3

/* ---- Continuing with more Tensor Operations to work better with data ---- */

DAY 3

If suppose, we have a tensor of random normals

↳ `torch::Tensor num_x = torch::randn({4,3,2});`

```
// Data Type
std::cout << num_x.dtype() << std::endl;

// Device (CPU or GPU)
std::cout << num_x.device() << std::endl;

// If CUDA is available
// if (auto_t_gpu --> Done if CUDA is available
   in system)

if (torch::cuda::is_available()) {
        auto t_gpu = num_x.to(torch::kCUDA);
        std::cout << t_gpu.device() << std::endl;
```

Tensor Index and Slicing

Think of a tensor like a huge bookshelf. Indexing is pointing to one specific book. Slicing is pointing to a whole shelf or a range of shelves

```
torch::Tensor t = torch::arange(0, 12).reshape({3,4});
// t looks like:
// 0  1  2  3
// 4  5  6  7
// 8  9  10 11

// Single Element
auto elem = t[1][2];          // Value at row 1 col 2 = 6
auto elem2 = t.index({1, 2}); // same
```

Torch Slicing

Slicing in LibTorch C++ is conceptually identical to NumPy or PyTorch in Python.

Slice() in C++ is equivalent to colon (:) in Python.

* In order to do slicing, we need

↳ using namespace torch::indexing;

```
auto row0 = t.index({0, Slice()});   // First row
auto col2 = t.index({Slice(), 2});   // Third column
auto sub = t.index({Slice(0,2), Slice(1,3)});
                                     // Rows 0-1, Cols 1-2
```

Understanding Math Operations For Tensors

So, How do we handle Element-Wise Operations

```
auto a = torch::Tensor ({1.0, 2.0, 3.0});
auto b = torch::Tensor ({4.0, 5.0, 6.0});
```

* We will do certain operations with these tensors (Mathematical)

```
auto add = a + b;
auto sub = b - a;
auto mul = a * b;
auto div = b / a;
auto pow = torch::pow(a, 2);
auto sq = torch::sqrt(b);
```